Introduction to Hydra

Configuration Management for ML Experiments

`YAML Config` `Config Groups` `instantiate()` `Multirun`

OVERVIEW

What is Hydra?

Hydra is an open-source framework from Meta Research for elegantly configuring complex Python applications—especially ML experiments.

Backed by: Meta fairseq, detectron2, hundreds of research codebases

```
Built on OmegaConf · Python 3.8+ · pip install hydra-core
```

Hierarchical Config

Compose YAML files like code modules

CLI Overrides

Change any param from the command line

Auto-Logging

Every run saved with full config & logs

Multirun Sweeps

Grid search with one --multirun flag

CORE CONCEPTS

Three Building Blocks

01 YAML Config Files

Define parameters in structured YAML. Hydra loads and merges them automatically.

```yaml
# conf/config.yaml
model:
  name: random_forest
  n_estimators: 100
random_state: 42
```

02 Config Groups

Swap entire config sections with one CLI flag. No code changes needed.

```
# One flag swaps everything:
python train.py model=svm
python train.py model=random_forest
```

03 instantiate()

Turn config directly into Python objects. No if/elif. No imports. No registries.

```
# Config → Python object:
clf = hydra.utils.instantiate(
    cfg.model
)
```

Your First Hydra App

conf/config.yaml

```yaml
random_state: 42
test_size: 0.2

model:
  name: logistic_regression
  max_iter: 1000
  n_estimators: 100
```

train.py

```python
import hydra
from omegaconf import DictConfig

@hydra.main(
    config_path="../conf",
    config_name="config"
)
def main(cfg: DictConfig):
    print(cfg.model.name)
    print(cfg.random_state)
```

CLI Overrides — dot notation changes any parameter:

```
python train.py
model.name=random_forest
```

```
python train.py model.max_iter=500
random_state=99
```

```
python train.py test_size=0.3
```

Config Groups — Swappable Components

Config Directory Structure

```
conf/
├── config.yaml          # defaults list
├── model/
│   ├── logistic_regression.yaml
│   ├── random_forest.yaml
│   └── gradient_boosting.yaml
└── preprocessing/
    ├── ordinal.yaml
    ├── onehot.yaml
    └── mean.yaml
```

conf/config.yaml — the defaults list

```yaml
defaults:
  - preprocessing: ordinal
  - model: logistic_regression
  - _self_
```

conf/model/random_forest.yaml

```yaml
_target_: sklearn.ensemble.RandomForestClassifier
n_estimators: 100
max_depth: null
random_state: ${random_state}
```

`model=random_forest`

swaps entire model config

`${random_state}`

interpolates from parent config

`_self_`

controls merge precedence

INSTANTIATE

instantiate() — Config as Code ✨

target tells Hydra what Python class to build. All other keys become constructor arguments.

YAML Config

```yaml
_target_: sklearn.ensemble
         .RandomForestClassifier
n_estimators: 100
max_depth: null
random_state: ${random_state}
```

instantiate
(cfg.model)

▶

Equivalent Python

```python
from sklearn.ensemble import (
    RandomForestClassifier
)

clf = RandomForestClassifier(
    n_estimators=100,
    max_depth=None,
    random_state=42
)
```

What instantiate() eliminates from your Python code:

- ❌ import statements for every class
- ❌ MODEL_REGISTRY dictionaries
- ❌ if/elif model selection chains
- ❌ custom build/dispatch functions

OmegaConf Interpolation

Reference values across config files using ${} syntax — changes propagate automatically.

conf/config.yaml

```
random_state: 42      # defined once

model:
  random_state: ${random_state}  # ←

preprocessing:
  random_state: ${random_state}  # ←

dataset:
  random_state: ${random_state}  # ←
```

Value Reference

```
random_state: ${random_state}
```

Pulls value from another config key

Nested Reference

```
lr: ${training.optimizer.lr}
```

Dot-path into nested config

String Interpolation

```
out: outputs/${model._target_}
```

Embed values inside strings

💡 Change random_state: 42 → random_state: 99 once, and every model and preprocessor updates automatically.

CLI OVERRIDES

Override Syntax — The Trifecta

`key=value`

MODIFY

Override an existing key. Raises error if the key doesn't exist.

Use case: Standard parameter tuning

```
python train.py model.n_estimators=500
```

`+key=value`

ADD

Add a new key not in the YAML. Raises error if the key already exists.

Use case: Extending config for one-off experiments

```
python train.py +model.warm_start=true
```

`++key=value`

FORCE

Force-set a key. Works whether or not the key already exists in YAML.

Use case: Automation scripts, CI/CD, Makefiles

```
python train.py ++model.n_estimators=500
```

Auto-Save & Reproducibility

Every Hydra run automatically creates a timestamped output directory — no manual logging needed.

outputs/ directory (created automatically)

```
outputs/
└── sklearn.ensemble.RandomForest/
    └── 2026-02-10_23-03-37/
        ├── .hydra/
        │   ├── config.yaml      # full config
        │   ├── overrides.yaml   # CLI args used
        │   └── hydra.yaml       # metadata
        ├── train.log            # all log output
        ├── metrics.json
        └── feature_importance.csv
```

Inspect any run

cat .hydra/overrides.yaml shows exactly what you ran

Reproduce results

Copy overrides back to CLI to re-run identically

Full config snapshot

config.yaml captures all resolved values including interpolations

Logging auto-captured

Python logging writes to .log file with zero setup

🎯 Reproducibility is the default. You don't earn it — Hydra gives it to you.